

Verification Pipeline

خط أنابيب التحقق والتدقيق

DAY_TRADE → Hardening → SSL → Execution Realism

من قواعد التداول اليومي إلى محرك التنفيذ الواقعي

Every audit, verdict, and feedback loop in one document.

كل عملية تدقيق وكل verdict وكل feedback loop في وثيقة واحدة.

1. Overview

١. نظرة عامة

Why a verification pipeline

لماذا خط أنابيب للتحقق

A trading system has many points where a silent bug can survive the test suite and only show up in production: a scaler that re-fits on the full dataset, a feature that is 95 % correlated with another, a strategy that only works in trending markets, a labeling gate that mis-classifies 80 % of bars as NEUTRAL, a slippage assumption that underestimates real execution by 3x. This pipeline catches each of those classes of bug with a dedicated, testable, automated tool.

slippage يقلل الواقع بـ 3 أضعاف. خط الأنابيب ده بيلاقي كل نوع منهم بأداة مخصصة قابلة للاختبار ومؤتمتة. trending markets, labeling gate يصنف 80% من الصفوف NEUTRAL خطأ، assumption يظهر إلا في الـ production: scaler يعيد الـ fit على كامل البيانات، ميزة مرتبطة 95% بميزة أخرى، استراتيجية نظام التداول مليء بنقاط ممكن فيها bug صامت ينجو من الـ test suite ولا

How to read this document

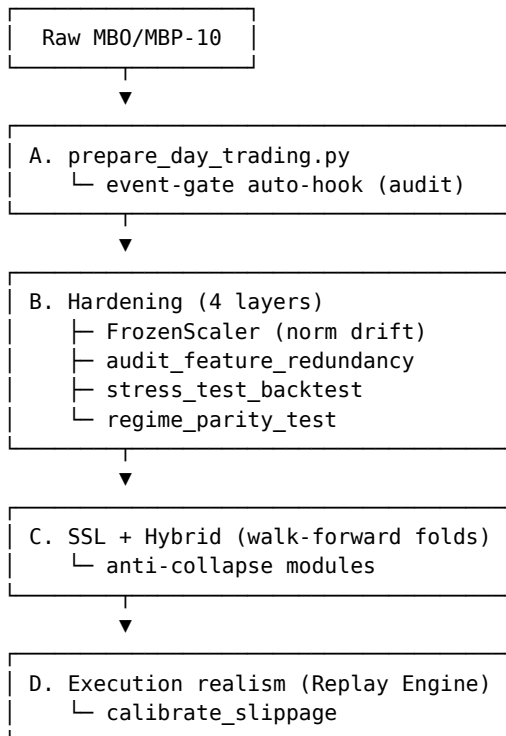
كيف تقرأ هذه الوثيقة

Each stage has: (1) what it produces, (2) the verification command, (3) the verdict it outputs, (4) what to do if the verdict is bad. Code blocks are copy-paste-runnable.

(٣) الـ verdict اللي بيطلع، (٤) إيه تعمل لو الـ verdict سيء. الـ code blocks جاهزة للنسخ والتشغيل مباشرة. كل مرحلة فيها: (١) المخرجات، (٢) أمر التحقق،

Pipeline at a glance

نظرة سريعة على المسار



2. Stage A — DAY TRADE pipeline

٢. المرحلة أ — خط أنابيب DAY_TRADE

What it does

ما الذي يفعله

Ingests raw Databento MBO/MBP-10 ticks, aggregates them into session-aware bars, enriches with regime + Kalman trend + Hawkes/absorption/Kyle microstructure signals, decides which bars are EVENTS via a six-component gate, and produces directional labels for the ones that pass.

+ regime, يقرر أي bars EVENTS عبر gate بـ ٦ مكونات، ويولد directional labels للـ bars التي عدت. في bars حسب الجلسة، يضيف Kalman trend + Hawkes/absorption/Kyle microstructure signals بيفراً ticks خام من (MBO أو MBP-10)، يجمعهم

Run it

كيف تشغله

```
python prepare_day_trading.py \
  --raw-mbo-dir raw/mbo \
  --output-dir combined/ \
  --symbol 6B
```

Auto-hook: event-gate audit

تلقائي Event Gate تدقيق Auto-hook:

After writing the features parquet, the pipeline runs the event-gate audit automatically and prints a verdict in line with the existing coverage diagnostics. The labels default to NEUTRAL on every bar where `is_event == 0`, so a starved gate is the single biggest cause of high-NEUTRAL training data. The hook makes that cause visible at write time instead of hiding it.

ده أكبر سبب لـ NEUTRAL عالي في training data. ال hook بيكشف السبب لحظة الكتابة بدل ما يبقى مخفي. coverage diagnostics. ال labels بتكون NEUTRAL على كل bar فيه `is_event=0`, فلو ال gate starved بعد ما ال pipeline يكتب ال features parquet، بيشغل تلقائياً `audit_event_gate` ويطبع ال verdict مع ال

Example output:

```
Dataset saved: combined/day_trading_features.parquet
Rows: 18,000 | Columns: 247
MBP coverage: bar_mean=82.3%, low(<30%)=4.2%
Training pool:
  train_event_flag=True : 3,820 rows (21.2%)
  LONG: 1,950 | SHORT: 1,870

Event gate: COMPONENT_DOMINATED (rate=12.4%) → combined/_audit_event_gate/
  ← high NEUTRAL traces to gate, not market behavior
  ← dominated_components: ['cvd_align_above_06']
```

Per-verdict response (event gate)

Verdict	Meaning	Action
HEALTHY	20-35% events, all regions clean	proceed to Stage B
LOW_RATE	10-20% events	loosen REGIME_EVENT_THRESHOLD
STARVED	<10% events	switch event_score to continuous_score
WARMUP_HEAVY	≥5% of bars killed by _zscore min_periods	drop warm-up bars from training set
COMPONENT_DOMINATED	the gate component fails >90% of bars	fix the source signal (eg. cvd_direction_pct fillna)
REGIME_STARVED	non-volatile regime <15% event rate	regime-specific threshold

3. Stage B — Operational hardening

٣. المرحلة ب — التصلب التشغيلي

Four layers, four classes of bug

أربع طبقات، أربع فئات من ال bugs

Each layer addresses one production-realistic failure mode documented in docs/PIPELINE_ISSUES_AUDIT.md.

كل طبقة تتعالج نوع واحد من ال failures الموثقة في docs/PIPELINE_ISSUES_AUDIT.md.

Layer	Tool	What it solves
Normalization drift	modules/trading_intel/hybrid/inference_runner.py (FrozenScaler)	Computes and stores mu/sigma fit at inference
Input redundancy	tools/audit_feature_redundancy.py	Drops 14 CVD / 12 ATR / 7 imbalance duplicate variants
Execution stress	tools/diagnostics/stress_test_backtest.py	Scenarios: baseline → mild → moderate → severe → extreme
Regime sensitivity	tools/diagnostics/regime_parity_test.py	Catches one-regime strategies that look profitable in agg

B.1 FrozenScaler — normalization drift

ب.١ FrozenScaler — انحراف التطبيق

Trains with mu/sigma fit only on the train slice. Inference loads a checkpoint and the scaler is frozen — every transform uses the stored stats; any fit() raises RuntimeError. There is no public path to accidentally re-fit at inference.

stats محفوظة؛ أي fit() يرمي RuntimeError. مفيش طريق عام لإعادة ال fit بالغلط أثناء ال inference. على train slice فقط. ال inference يحمل checkpoint وال scaler مجمد — كل transform يستخدم التدريب ببيات ال mu/sigma

```
from modules.trading_intel.hybrid.inference import (
    load_hybrid_for_inference, predict_one,
)
model, scaler, cfg = load_hybrid_for_inference("runs/fold_01")
pred = predict_one(model, scaler, latest_features_dict)
# scaler.fit(...) → RuntimeError
```

B.2 Feature-redundancy audit

ب.٢ تدقيق التكرار في الميزات

Builds a Pearson correlation graph at threshold τ (default 0.9), takes connected components, marks all but the most-predictive feature in each cluster as redundant. Writes redundancy_summary.json with a drop_list consumed by both fold runners via --drop-features-from-audit.

redundancy_summary.json فيه drop_list يستهلكه ال fold runners عبر --drop-features-from-audit. باخد connected components، يحتفظ بأقوى ميزة في كل cluster ويعتبر الباقي تكرار. ييكتب بييني correlation graph بين الميزات عند 0.9 default)

```
python tools/audit_feature_redundancy.py \
    --features combined/day_trading_features.parquet \
    --output audits/redundancy --threshold 0.9
```

B.3 Stress test

ب.٣ اختبار الإجهاد التنفيذي

Re-runs the strict backtest under five execution-quality scenarios from baseline to extreme (latency bars + slippage pips both monotonically worsening). Verdict from final-fold Sharpe stability across scenarios.

extreme (latency bars + slippage pips) ييزيدوا (monotonically). ال verdict بناء على ثبات Sharpe عبر السيناريوهات. ييعيد ال backtest تحت 0 سيناريوهات تنفيذ متدرجة من baseline لـ slippage pips

Verdict	Sharpe stability
ROBUST	All 5 scenarios profitable
ACCEPTABLE	Worst case still ≥ 0.5 Sharpe
FRAGILE	Severe scenario breaks the edge
BROKEN	Mild scenario already breaks the edge

B.4 Regime parity

ب.٤ تكافؤ ال regime

Splits trades by categorical regime AND by realised-vol quartile, computes per-subset Sharpe/PF/DD and a coefficient of variation across subsets. Catches strategies that secretly depend on one regime to be profitable.

coefficient of variation عبر ال subsets. يكتشف الاستراتيجيات التي خفية بتعتمد على regime واحد علشان تكون رابحة. حسب ال regime (تصنيفي) وحسب volatility quartile، ويحسب Sharpe/PF/DD لكل subset مع subset variation بيقسم الصفقات

```
python tools/diagnostics/regime_parity_test.py \
  --trades-csv backtests/baseline_strict/trades.csv \
  --output      diagnostics/regime --min-trades 30
```

4. Stage C — SSL training

٤. المرحلة ج — تدريب ال SSL

Walk-forward + anti-collapse

مضاد الانهيار + Walk-forward

The SSL encoder + hybrid head are trained per fold on an expanding train window with a forward holdout. Four modules (individually toggleable) protect against the documented failure modes of multi-task SSL.

أربع modules (كل واحدة قابلة للتفعيل المستقل) بتحمي من failure modes الموثقة في multi-task SSL. ال SSL encoder + hybrid head بيتدربوا per fold على expanding train window مع forward holdout.

Module	Failure it prevents	Activation flag
Fixed Simplex ETF	direction-head collapse to single class	--use-simplex-etf
Orthogonal Representation	feature-collapse (all heads share one subspace)	--ortho-weight W
DB-MTL balancer	small tasks dominated by large ones	--use-dbmtl
Differentiable Sharpe	loss is proxy for accuracy, not return	--sharpe-weight W

Run a fold (all four enabled):

```
python tools/run_walk_forward_fold_enhanced.py \  
  --fold-id 1 \  
  --combined-features combined/day_trading_features.parquet \  
  --ssl-embeddings embeddings/ssl_macro.parquet \  
  --embedding-valid embeddings/ssl_valid_mask.parquet \  
  --train-start 2021-01-01 --train-end 2023-12-31 \  
  --test-start 2024-01-01 --test-end 2024-06-30 \  
  --output-dir runs/fold_01 \  
  --drop-features-from-audit audits/redundancy/redundancy_summary.json \  
  --use-simplex-etf \  
  --ortho-weight 0.05 \  
  --use-dbmtl \  
  --sharpe-weight 0.1
```

Aggregate across folds:

```
python tools/aggregate_walk_forward.py \  
  --runs-dir runs/ \  
  --output aggregated_metrics.json
```

5. Stage D — Execution realism

٥. المرحلة د — واقعية التنفيذ

Replay engine + slippage calibration

Replay engine + slippage معايرة ال

Bar-level backtest assumes constant 0.5-pip slippage and integer-bar latency — fine for ranking strategies, blind to what a 50-lot order does to the level-2 book. Stage D fixes that with two stand-alone tools that form a closed feedback loop.

عن تأثير أمر lot 50 على L2 book. المرحلة د بتحل ده بأداتين مستقلتين بتشكلا closed feedback loop. ال bars يفترض slippage ثابت 0.5 pip و latency بـ bars صحيحة — كويس لمقارنة استراتيجيات، لكن أعمى ال backtest على مستوى

D.1 Replay engine

١.١ محرك الإعادة (Replay Engine)

modules/replay_engine/ — event-by-event MBP-10 simulator. Walks the book level-by-level on each market order, samples lognormal latency between decision and arrival, logs every fill with the LOB snapshot at fill time. Pure separation: snapshots are frozen dataclasses; fill_market_order is a pure function; only the orchestrator owns time and RNG.

modules/replay_engine/ — محاكي MBP-10 على ال. بيمشي على ال book حدث-حدث. decision وال arrival بين ال lognormal latency بيسحب market order، مع fill بيسجل كل arrival وال decision بين ال lognormal latency بيسحب market order، snapshots frozen dataclasses: fill_market_order pure function. الفصل صافي: ال snapshot ال RNG. هو الوحيد اللي بيملك الوقت وال orchestrator ؛ ال function.

```
python tools/run_replay_backtest.py \
--signals backtests/baseline/signals.csv \
--mbp raw/mbp_10.parquet \
--output replay_results/baseline \
--tick-size 0.0001 \
--latency-mean-us 5000 --latency-jitter-us 2000 \
--slip-base-ticks 0.5 --slip-mid-ticks 1.5
```

Outputs

- execution_log.jsonl — one JSON object per fill (snapshot included)
- replay_summary.json — aggregate latency/slippage/fill-quality stats
 - execution_log.jsonl — fill (مع ال snapshot) واحد بيمثل JSON كل سطر
 - replay_summary.json — fill وجودة ال slippage وال latency إحصاءات تجميعية للـ

D.2 AdaptiveSlippage — three regions

٢.١ AdaptiveSlippage — ثلاث مناطق

Region	Condition	Predicted ticks
A — sub-L1	$\text{size} / \text{L1_depth} \leq 0.10$	base_ticks (flat)
B — linear	$0.10 < \text{ratio} \leq 1.0$	$\text{base} + t \cdot (\text{mid} - \text{base}), t = (\text{ratio} - 0.10) / 0.90$
C — walking	$\text{ratio} > 1.0$	$\text{mid} + \text{extra_per_level} \cdot (\text{levels_consumed} - 1)$

D.3 Calibration loop

٣.١ حلقة المعايرة

Reads execution_log.jsonl, splits events by region, refits each constant via OLS-through-origin on its own subset, then applies safety clamps (non-negativity + base ≤ mid). Writes slippage_config_suggested.json as a drop-in for the next replay run.

(عدم سالبية + mid $\leq$ base). يكتب slippage_config_suggested.json كـ drop-in للـ replay اللي بعده.
الـ region، يعيد fit كل constant عبر OLS-through-origin على subset بتاعها، ثم يطبق safety clamps
بيقرأ execution_log.jsonl، يقسم الـ events حسب

```
python tools/calibrate_slippage.py \  
  --log      replay_results/baseline/execution_log.jsonl \  
  --output   replay_results/baseline/calibration \  
  --tick-size 0.0001 \  
  --current-base 0.5 --current-mid 1.5 --current-extra 1.0
```

Closed feedback loop

```
run_replay_backtest → execution_log.jsonl  
  ↓  
calibrate_slippage → slippage_config_suggested.json  
  ↓  
run_replay_backtest --slip-base X --slip-mid Y --slip-extra Z  
  ↓  
(repeat until verdict = WELL_CALIBRATED)
```


6. Verdict cascades — all in one place

٦. شجرات ال verdicts — في مكان واحد

Every audit emits a categorical verdict. Use this table to decide whether the run is shippable.

كل audit ييخرج verdict تصنيفي. الجدول ده بيقولك إذا ال run جاهز لل production أم لا.

Audit	OK	Marginal	Block
Event gate	HEALTHY	LOW_RATE / WARMUP_HEAVY	STALE / COMPONENT_DOMINATED
Redundancy	drop_list applied	drop_list large but applied	drop_list not applied
Stress test	ROBUST	ACCEPTABLE	FRAGILE / BROKEN
Regime parity	ROBUST	ACCEPTABLE	UNSTABLE / REGIME_BIAS / INSUFFICIENT_DATA
Slippage calibration	WELL_CALIBRATED	OVER_CALIBRATED (safe)	UNDER_CALIBRATED (dangerous)

Why some 'Block' verdicts are tolerable

لماذا بعض verdicts ال Block محتملة

- OVER_CALIBRATED (slippage) — model is conservative; the system loses some marginal trades that would have been profitable but will never put on a trade it shouldn't.
- WARMUP_HEAVY (event gate) — fixable by dropping the warm-up bars from the training set; the live system never sees them.
- FRAGILE (stress) — acceptable as a documented risk only if the broker is known to provide consistent execution.
 - OVER_CALIBRATED (slippage) — النظام بيخسر بعض الصفقات ال conservative ال model ؛ لكن لن يدخل في صفقة كان مفروض ميدخلش فيها marginal.
- WARMUP_HEAVY (event gate) — ال training set من warm-up bars قابل للإصلاح بطرد ال live ؛ مش بيشوفهم أصلا system.
- FRAGILE (stress) — معروف بتنفيذ ثابت broker مقبول كمخاطرة موثقة فقط لو ال

7. End-to-end runbook

٧. خطوات التشغيل من البداية للنهاية

Run these in order. Each command's verdict gates the next.

شغلهم بالترتيب. verdict كل أمر يحدد إذا اللي بعده يشتغل.

Step 1 — Build features + auto-audit

```
python prepare_day_trading.py \  
  --raw-mbo-dir raw/mbo --output-dir combined/ --symbol 6B  
# expect: □/△□/□ Event gate verdict on stdout
```

Step 2 — Feature redundancy audit

```
python tools/audit_feature_redundancy.py \  
  --features combined/day_trading_features.parquet \  
  --output audits/redundancy --threshold 0.9
```

Step 3 — Walk-forward folds (loop over fold_id)

```
for FOLD in 1 2 3 4 5 6 7 8; do  
  python tools/run_walk_forward_fold_enhanced.py \  
    --fold-id $FOLD \  
    --combined-features combined/day_trading_features.parquet \  
    --ssl-embeddings embeddings/ssl_macro.parquet \  
    --embedding-valid embeddings/ssl_valid_mask.parquet \  
    --output-dir runs/fold_$FOLD \  
    --drop-features-from-audit audits/redundancy/redundancy_summary.json \  
    --use-simplex-etf --ortho-weight 0.05 \  
    --use-dbmtl --sharpe-weight 0.1  
done  
python tools/aggregate_walk_forward.py --runs-dir runs/ --output runs/agg.json
```

Step 4 — Stress test

```
python tools/diagnostics/stress_test_backtest.py \  
  --features combined/day_trading_features.parquet \  
  --output diagnostics/stress  
# expect: verdict ROBUST or ACCEPTABLE
```

Step 5 — Regime parity

```
python tools/diagnostics/regime_parity_test.py \  
  --trades-csv backtests/baseline_strict/trades.csv \  
  --output diagnostics/regime  
# expect: verdict ROBUST or ACCEPTABLE
```

Step 6 — Replay backtest (execution realism)

```
python tools/run_replay_backtest.py \  
  --signals backtests/baseline_strict/signals.csv \  
  --mbp raw/mbp_10.parquet \  
  --output replay_results/baseline \  
  --tick-size 0.0001
```

Step 7 — Calibrate slippage

```
python tools/calibrate_slippage.py \  
  --log replay_results/baseline/execution_log.jsonl \  
  --output replay_results/baseline/calibration \  
  --tick-size 0.0001  
# expect: verdict WELL_CALIBRATED after 1-2 iterations
```

Step 8 — Verify in inference mode

```
python -c "  
from modules.trading_intel.hybrid.inference import (  
    load_hybrid_for_inference, predict_one,  
)  
m, sc, cfg = load_hybrid_for_inference('runs/fold_01')  
print('FrozenScaler intact:', sc.is_frozen())  
"
```

If every step ends with a green verdict, the build is shippable. If any step blocks, fix that step before moving on — don't override.

جاهز للـ production. لو أي خطوة بلوك، أصلح المرحلة دي قبل ما تمشي للي بعدها — متعملش override.
لو كل خطوة طلعت verdict أخضر، الـ build